Step 2 — Backend placeholder (API + worker)

Part of **S0.1.1 Monorepo with Docker Compose** (GitHub issue #2).

Goal: a minimal FastAPI app and a minimal ARQ worker that both run from one Docker image. They do almost nothing yet; they exist so that `docker compose up` (step 5) has real services to start. Real features arrive in later stories (S0.2.1 health checks, S1.3.1 ingestion worker, and so on).

You'll touch these files, all already created and empty:

```
docqa-backend/
├── .dockerignore          (already filled in)
├── Dockerfile
├── requirements.txt
└── app/
    ├── __init__.py        (stays empty)
    ├── main.py
    └── workers/
        ├── __init__.py    (stays empty)
        └── tasks.py
```

The two `__init__.py` files stay empty. They only mark `app` and `app.workers` as Python packages so imports like `app.main` work.

2.1 `requirements.txt`

```
fastapi>=0.115
uvicorn[standard]>=0.30
arq>=0.26
```

Package	Why
`fastapi`	The web framework for the API
`uvicorn[standard]`	The server that runs FastAPI; `[standard]` adds faster HTTP parsing and `--reload` file watching
`arq`	The job queue on Redis (decision D3 in issue #1)

These are minimum versions, not exact pins. Once everything works, you can pin exact versions with `pip freeze` if you want fully repeatable builds.

2.2 `app/main.py`

```
from fastapi import APIRouter, FastAPI

app = FastAPI(title="DocQA API")

api_router = APIRouter(prefix="/api/v1")


@api_router.get("/health")
async def health() -> dict[str, str]:
    return {"status": "ok"}


app.include_router(api_router)
```

What it does:

- `app` is the FastAPI application that uvicorn serves.
- Every route goes on `api_router`, so they all live under `/api/v1`, matching the architecture doc.
- `/api/v1/health` returns `{"status": "ok"}`. In S0.2.1 it becomes a real check of Postgres, Redis and Ollama.

2.3 `app/workers/tasks.py`

```
import os

from arq.connections import RedisSettings


async def ping(ctx: dict) -> str:
    """Placeholder task that proves the worker can receive and run jobs."""
    return "pong"


class WorkerSettings:
    functions = [ping]
    redis_settings = RedisSettings.from_dsn(os.environ.get("REDIS_URL",
"redis://localhost:6379/0"))
```

What it does:

- ARQ starts from a `WorkerSettings` class. You run it with `arq app.workers.tasks.WorkerSettings`.
- `functions` lists the tasks this worker can run. `ping` is a placeholder; `ingest_document` replaces it in S1.3.1.
- Every task receives `ctx` first, a dict ARQ uses for shared things such as the Redis connection (and later your DB session).
- `REDIS_URL` comes from the environment:
    - in Docker it will be `redis://redis:6379/0`, where `redis` is the compose service name

- locally it falls back to `localhost`

2.4 `Dockerfile`

```
FROM python:3.12-slim

# Don't write .pyc files; print logs immediately instead of buffering them
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /code

# Install dependencies first so this layer is cached until requirements.txt
changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

EXPOSE 8000

# Default command runs the API; docker-compose overrides it for the worker
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why it's written this way:

- **One image, two services.** The API and the worker share the same code and dependencies. Compose runs the image twice with different commands: `uvicorn …` for `api` and `arq …` for `worker`.
- **`requirements.txt` is copied before `app/`.** Docker caches each step. Editing code then reuses the cached `pip install` layer, so rebuilds take seconds.
- **`--host 0.0.0.0`** makes uvicorn listen on all interfaces inside the container. The default, `127.0.0.1`, can't be reached from outside the container.
- **`.dockerignore`** already excludes `.venv`, so your Windows venv is never copied into the Linux image.

> This updates the earlier guidance. I first said to leave out `CMD`; a default `CMD` is more useful because `docker run` on the image then starts the API on its own.

2.5 Verify locally (without Docker)

Run these in PowerShell from `D:\DocReaderQA\docqa-backend`.

a) Activate the venv and install dependencies

```
.\.venv\Scripts\Activate.ps1
python --version          # should print Python 3.12.x
pip install -r requirements.txt
```

If activation fails with "*running scripts is disabled on this system*", run this once and try again:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

b) Run the API

```
uvicorn app.main:app --reload
```

Then open:

- http://localhost:8000/api/v1/health — should show `{"status":"ok"}`
- http://localhost:8000/docs — FastAPI's interactive API docs

Stop the server with `Ctrl+C`.

c) Run the worker (needs Redis)

Start a temporary Redis in Docker:

```
docker run -d --name docqa-redis -p 6379:6379 redis:7-alpine
```

In one terminal (venv active), start the worker:

```
arq app.workers.tasks.WorkerSettings
```

It should log that it is starting with the function `ping`.

To send it a job, save this as a throwaway script **outside the repo**, e.g. `D:\scratch\send_ping.py`:

```
import asyncio

from arq import create_pool
from arq.connections import RedisSettings


async def main():
    pool = await create_pool(RedisSettings())
    job = await pool.enqueue_job("ping")
    print(await job.result(timeout=10))


asyncio.run(main())
```

Run it in a second terminal with the venv active:

```
python D:\scratch\send_ping.py
```

It should print `pong`, and the worker terminal should log the job.

Clean up so the temporary Redis doesn't clash with the compose Redis on port 6379 in step 5:

```
docker rm -f docqa-redis
```

> If the worker misbehaves on Windows (for example around `Ctrl+C`/signal handling), skip this local test and verify the worker in Docker in step 5 instead. It runs on Linux there, which is how it will always run.

d) Build and run the image

```
docker build -t docqa-backend .
docker run --rm -p 8000:8000 docqa-backend
```

Open http://localhost:8000/api/v1/health again, then stop the container with `Ctrl+C`.

Done when

- [ ] `requirements.txt`, `app/main.py`, `app/workers/tasks.py` and `Dockerfile` are filled in
- [ ] `uvicorn app.main:app --reload` serves `{"status":"ok"}` at `/api/v1/health`
- [ ] The worker runs the `ping` job and returns `pong` (locally, or deferred to step 5)
- [ ] `docker build` succeeds and the container serves the health endpoint
- [ ] `docker rm -f docqa-redis` done

Suggested commit

```
git add docqa-backend
git commit -m "Add backend placeholder API and ARQ worker (refs #2)"
```

`refs #2` links the commit to the S0.1.1 issue without closing it. S0.1.1 isn't finished until steps 3–7 are done too.

Common problems

Symptom	Cause	Fix
`ModuleNotFoundError: No module named 'app'`	Command run from the wrong folder	Run from `docqa-backend`, not `docqa-backend\app`

Symptom	Cause	Fix
`uvicorn` / `arq` not recognised	venv not active	`.\.venv\Scripts\Activate.ps1`
Port 8000 already in use	Another server still running	Stop it, or use `--port 8001`
Worker can't connect to Redis	Temporary Redis not running	`docker ps` should list `docqa-redis`
`docker build` is slow every time	`requirements.txt` copied after `app/`	Keep the order shown above

Next: step 3, `.env.example`.